

P2003R0

Fixing Internal and External Linkage Entities in Header Units

Published Proposal, 2020-01-13

This version:

<http://wg21.link/p2003r0>

Author:

[Michael Spencer](#) (Apple)

Audience:

EWG, SG2

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Currently internal linkage entities (such as `static inline` functions) and external linkage entities are allowed in header units, but there are two problems with how they are currently specified. First, C++ currently requires there to be a single header unit for a given header or source file ([module.import]/5). This means that regardless of how many times or from where a header is imported, there must be a single copy of any internal or external linkage entities within it. The second is that it is ill-formed to use internal linkage entities outside of that header unit. This is problematic as it is quite common for headers that want to be compatible with C to use `static inline` for inline functions, as C's version of inline works differently.

This paper explores these problems and what current implementations do in these cases, and fixes it to be more compatible with current/expected usage by defining a model compatible with those implementations, which addresses the issues raised by US133 and US134.

Table of Contents

- 1 **Effects of This Paper**
- 2 **The Problem**
- 3 **The Solution**
 - 3.1 Undefined Behavior
- 4 **Ship Vehicle**

§ 1. Effects of This Paper

Status Quo	This Paper
header.h <pre>#ifndef HEADER_H #define HEADER_H int ext(); // Has address identity. static inline int abs(int V) { return V >= 0 ? V : -V; } int a(int V) { // ✓ return abs(V); } inline int b(int V) { // ✓ return abs(V); // ✓ } // Guaranteed to be called once. static int glob = ext(); // ✓ struct c { int var; }; // ✓ int ext_glob = ext(); // ✓ #endif</pre> func.cpp	header.h <pre>#ifndef HEADER_H #define HEADER_H int ext(); // Does not have address identity. static inline int abs(int V) { return V >= 0 ? V : -V; } int a(int V) { // ⚠ ODR, linker error return abs(V); } inline int b(int V) { // ✓ return abs(V); // ⚠ ODR, different def } // Multiple definitions. static int glob = ext(); // ✓ struct c { int var; }; // ✓ int ext_glob = ext(); // ⚠ ODR, linker error #endif</pre> func.cpp

Status Quo	This Paper
<pre> import "header.h" int func(int V) { V += glob; // ✗ V += ext_glob; // ✓ V += a(V); // ✓ V += b(V); // ✓ return abs(V) + 4; // ✗ } inline int func2(int V) { V += glob; // ✗ V += ext_glob; // ✓ V += a(V); // ✓ V += b(V); // ✓ return abs(V) + 4; // ✗ } </pre>	<pre> import "header.h" int func(int V) { V += glob; // ✓ V += ext_glob; // ✓ V += a(V); // ✓ V += b(V); // ✓ return abs(V) + 4; // ✓ } inline int func2(int V) { V += glob; // ☠ ODR, different def V += ext_glob; // ✓ V += a(V); // ✓ V += b(V); // ✓ return abs(V) + 4; // ☠ ODR, different def } </pre>
m.cpp	m.cpp
<pre> export module m; import "header.h" export int mfunc(int V) { V += glob; // ✗ V += ext_glob; // ✓ V += a(V); // ✓ V += b(V); // ✓ return abs(V) + 4; // ✗ } export inline int mfunc2(int V) { V += glob; // ✗ V += ext_glob; // ✓ V += a(V); // ✓ V += b(V); // ✓ return abs(V) + 4; // ✗ } </pre>	<pre> export module m; import "header.h" export int mfunc(int V) { V += glob; // ✓ V += ext_glob; // ✓ V += a(V); // ✓ V += b(V); // ✓ return abs(V) + 4; // ✓ } // These two would be ill-formed under P1815. export inline int mfunc2(int V) { V += glob; // ☠ ODR, different def V += ext_glob; // ✓ V += a(V); // ✓ V += b(V); // ✓ return abs(V) + 4; // ☠ ODR, different def } </pre>

Legend:

- ✓ Works as expected.
- ✗ Ill-formed.
- ☠ Undefined behavior

§ 2. The Problem

[module.import]/5

... Two *module-import-declarations* import the same header unit if and only if their *header-names* identify the same header or source file. [...] A declaration of a name with internal linkage is permitted within a header unit despite all declarations being implicitly exported. If such a declaration declares an entity that is odr-used outside the header unit, or by a template instantiation whose point of instantiation is outside the header unit, the program is ill-formed.

This requires a single header unit for each imported header and prohibits using internal linkage entities from that header unit.

The first problem with this is that requiring a single header unit in a program for a given header file borders on unimplementable. Some compiler invocation has to produce that header unit, but there's no reasonable way to decide which one. For example, two different libraries developed by two unrelated groups use a C library libO. Both of them `import <O/functionality.h>` which includes a static local variable with a dynamic initializer which observes how many times it was called. A third group of developers comes along and tries to use both of these libraries in the same program. How do you ensure that static local variable is only initialized once? libO is a C library written before C++20 modules existed, and so it doesn't expect to need to build the header unit for O/functionality.h, so it can't be responsible. The two other libraries don't even know about each other, so neither one of those can assume it can build it. The third group doesn't even know this header exists as it's an internal implementation detail of the other two libraries.

The end result is there's no sane way to implement the requirement that there's a single header unit for each imported header or source file.

The second problem is that `static inline` functions are pretty common in headers for two main reasons:

- C's inline semantics are different from C++, and would require a non-inline definition somewhere.
- They provide ABI isolation as you'll never get a different version from what the compiler saw. This means you can totally change how they work without fear of breaking ABI.

A search on github finds thousands of uses of static inline in C headers including projects such as Wayland, OpenSSL, Clang builtins, mono. This is also used pervasively on Apple platforms in the form of `NS_INLINE` which is defined as `static inline`. Additionally, almost all inline functions in libc++ on Apple platforms are `static inline` for ABI isolation reasons, even member functions (via an extension). Even if both of these cases could be changed, just fixing these isn't enough. There are still at least 10s of thousands of instances of `static inline` in the wild, and it would be unfortunate if they were not usable as header units.

§ 3. The Solution

Clang modules currently supports internal linkage entities by simply copying them into whatever translation unit imports the module. This works quite well as it's the same semantics you would get by textually including the header the Clang module was build from. However, this may not be the best way to specify it.

Instead, this paper proposes making two changes. The first is to say that instead of there being a single header unit for each header or source file, there may be multiple, and it's unspecified which one is imported. The second is that you may

name imported internal linkage entities from header units with the same ODR implications as today.

The first item fixes the first problem above about not knowing who's responsible for building the header unit, as now every TU that imports the header can build its own. This also fixes the issue with external linkage entities as you'll likely end up with multiple definitions of them, which would be an error. Note that specifying it this way does not require that each importing TU build its own header unit. A sufficiently advanced build system could still build a single one for the entire program.

The second item fixes the second problem above by removing the restriction on naming internal linkage entities. This is implementable by either copying the definition into the importing TU, or by doing linkage promotion, as the spec allows there to be one or more instances of any internal linkage entity in this case.

§ 3.1. Undefined Behavior

This paper introduces two different types of UB. The first is that by allowing multiple header units to exist for the same header or source file without disallowing certain external linkage entities in header units, it allows for violating the rule that *Every program shall contain exactly one definition of every non-inline function or variable ...* While a violation of this rule does not require a diagnostic, in the majority of situations a user will get a multiple definition error. This could be made ill-formed instead, but it was felt it was simpler at this time to stay closer to header semantics.

The second type of UB is a more inconspicuous case. The one definition rule (ODR) allows for multiple definitions of inline variables and functions, but they must be the "same". Any inline definition that references an internal linkage entity is a ODR violation waiting to happen. However, `static inline` functions are extremely common today, and a large portion of C headers would not be usable as header units if they were not allowed to be referenced. As header units exist purely to support existing code and code that will likely never move to modules (like C code), it would significantly harm modules adoption to not allow this. The consequences are the same as using `static inline` from a textual header today. We're not introducing any new UB, just not stopping people from hitting it.

§ 4. Ship Vehicle

This paper is targeting C++20 as many C headers are not importable without it.

§ 5. Wording

TBD